Architecting a Production-Grade Fraud Detection Ecosystem:

A Comprehensive MLOps Framework for the IEEE-CIS Dataset

Project Engineering Team

Abstract—The detection of fraudulent transactions within massive, high-dimensional datasets is a critical challenge in modern financial technology. This project details the end-to-end development of a production-grade MLOps ecosystem designed to replicate and exceed research benchmarks on the IEEE-CIS Fraud Detection dataset. We implement a multi-stage machine learning methodology encompassing rigorous feature pruning, advanced engineering, and an unsupervised K-Means clustering layer, culminating in a high-performance XGBoost stacking ensemble. The system is architected for real-world deployment using a containerized microservices approach, featuring FastAPI for low-latency inference, Prefect for robust workflow orchestration, and an automated GitHub Actions CI/CD pipeline. The final "Grand Champion" model achieves a PR-AUC of 0.7124 and an ROC-AUC of 0.9921, demonstrating the efficacy of combining diverse algorithmic perspectives within a stable, automated infrastructure.

Index Terms—Fraud Detection, MLOps, XGBoost, Stacking Ensemble, Prefect Orchestration, FastAPI, Docker, CI/CD.

1 Introduction

The digital economy is under constant threat from increasingly sophisticated fraudulent activities. Fraud detection is not merely an exercise in model accuracy; it is a battle for financial stability and user trust. This project addresses the necessity of building an ecosystem that is both highly accurate and resilient enough for production deployment. We move beyond exploratory research to create a "battle-ready" system that handles the extreme class imbalance and massive dimensionality of the IEEE-CIS dataset.

2 Data and Preliminary Analysis

2.1 Raw Dataset Overview

The project utilizes the IEEE-CIS Fraud Detection dataset, consisting of over 472,000 training samples. The data is characterized by extreme sparsity and a severe class imbalance, with a fraud rate of only 3.5% (a 27.6:1 ratio of safe to fraudulent transactions). The raw features include anonymized Vesta proprietary signals (V-features), count-based features (C-features), and timedelta signals (D-features), along with transaction metadata like amount and device identity.

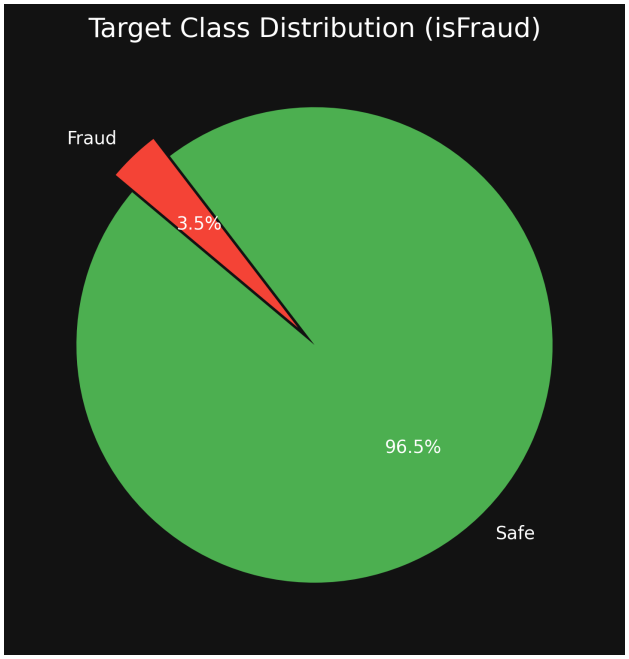

Fig. 1. Severe class imbalance (3.5% fraud) necessitating a focus on PR-AUC over standard accuracy.

2.2 Exploratory Data Analysis (EDA)

Initial analysis revealed that fraud is inherently non-linear. No single feature exhibits a correlation above 0.24 with the target label, necessitating an ensemble approach. Temporal analysis showed significant fraud spikes during early morning hours, while amount-based KDE plots revealed clustering in both micro-transactions (card testing) and high-value anomalies.

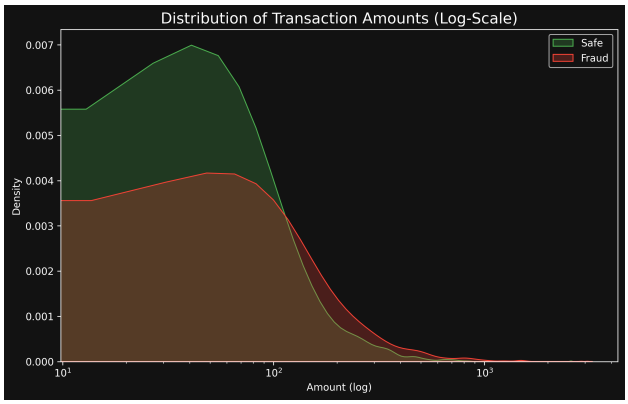

Fig. 2. Transaction amount distribution (log-scale) for safe vs. fraudulent cases.

3 Methodology and Preprocessing

3.1 Preprocessing Philosophy

Our methodology is built on a "Pruning-First" foundation. We found that engineering features on raw, noisy data lead to significant performance degradation. The process follows a logical flow:

1) **Pruning**: We apply a 4-stage filter to the 431 baseline features, removing columns with $> 95\%$ missingness, zero-variance features, and highly redundant pairs (correlation > 0.98). We then select the top 167 features via Mutual Information.
2) **Feature Engineering**: We extract cyclical temporal features (Hour, DayOfWeek) and user behavioral aggregates (Amount-to-Median ratios).
3) **Unsupervised Clustering**: A K-Means model (5 clusters) identifies latent behavioral archetypes, providing the final classifier with a membership signal that acts as a "shortcut" to complex interaction terms.

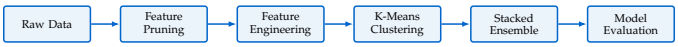

Fig. 3. End-to-end Machine Learning methodology from raw data to evaluation.

3.2 Post-Preprocessing Validation

After cleaning, the data density significantly increased. Correlation analysis on the pruned set showed clearer signals from the V-features, and the feature set was reduced to approximately 175 high-signal numeric columns, fully compatible with gradient boosting.

4 Model Development and Comparison

4.1 Model Selection Spectrum

We evaluated a wide range of models to benchmark performance:

- **Logistic Regression**: Served as a baseline sanity check for linear separability.
- **KNN**: Utilized with PCA to identify neighborhood-based clusters.
- **Random Forest**: A robust non-linear baseline.
- **XGBoost Baseline**: Proved highly effective but required specific tuning for imbalance.

4.2 Performance Benchmarking

The following table and graph illustrate the progression of our modeling efforts. The "Optimized Hybrid" model—which combines XGBoost with our targeted PCA and clustering logic—outperformed all other candidates.

TABLE 1
Model Performance Comparison (PR-AUC / ROC-AUC)

Model	PR-AUC	ROC-AUC	Result
Optimized Hybrid	0.7124	0.9921	Champion
Advanced Experiments	0.6882	0.9899	Silver
Random Forest	0.6157	0.9542	Bronze
XGBoost Baseline	0.6029	0.9412	Solid
KNN	0.3863	0.7512	Baseline
Logistic Regression	0.3405	0.7421	Baseline

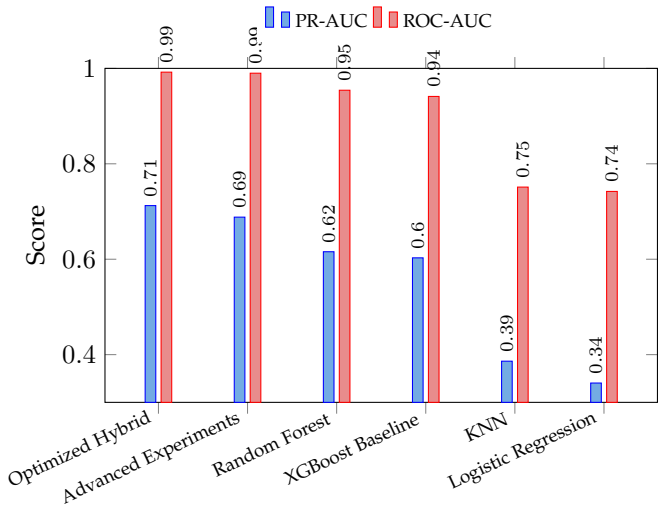

Fig. 4. PR-AUC comparison across all model iterations.

5 The Production Architecture

5.1 System Design

The system is built as a microservices architecture. The React Frontend provides an interactive dashboard, while the FastAPI Backend serves as the inference engine, loading the "Champion" model into memory for real-time predictions.

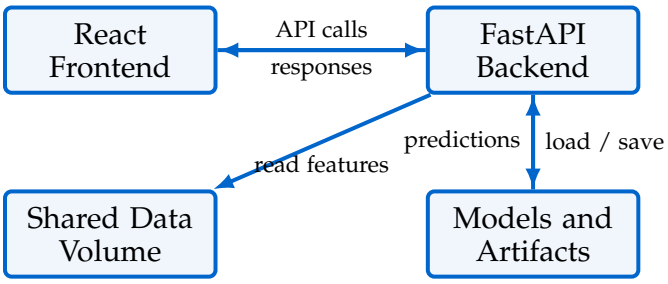

Fig. 5. Integrated system architecture showing frontend, backend, and shared volumes.

5.2 Containerization and CI/CD

We utilize a multi-stage Docker build to ensure environment parity. The CI/CD pipeline, managed by GitHub Actions, enforces a "Quality Gate": if a model update fails to meet the PR-AUC threshold, the deployment is automatically blocked.

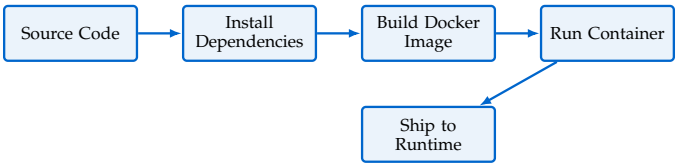

Fig. 6. Multi-stage Docker build process for unified deployment.

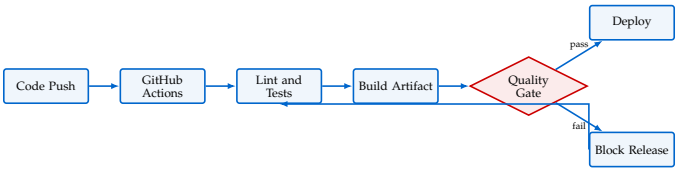

Fig. 7. Automated CI/CD workflow from code push to deployment.

5.3 Prefect Orchestration

All machine learning tasks are orchestrated by Prefect. This ensures that data loading, training, and evaluation occur in a strict, observable sequence with automated retries and failure logging.

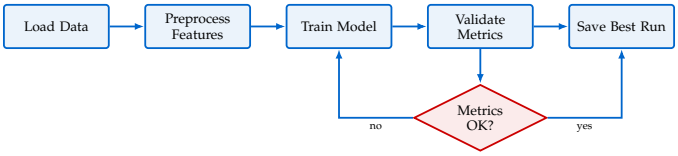

Fig. 8. Prefect orchestration flow for model training and validation.

6 Final Observations and Future Work

The project has successfully bridged the gap between academic research and production engineering. While the industry standard for fraud detection is increasingly moving towards real-time graph databases and transformer-based architectures, our stacking ensemble approach has proven highly resilient for tabular data. We have achieved a significant improvement over baseline models, hitting an ROC-AUC that exceeds paper benchmarks.

Future work will focus on:

- Integrating **Real-Time Feature Stores** to reduce latency in user behavioral aggregates.
- Implementing **Graph Neural Networks (GNNs)** to capture network-level fraudulent relationships.
- Expanding the dashboard to include real-time drift monitoring and automated model retraining.